The Semantic Commit Message Bible

A complete reference for writing meaningful, machine-readable git history using the Conventional Commits specification.

Every commit you write is a message to the future — to teammates, to CI tools, and to yourself six months from now. Semantic commits turn a flat list of cryptic one-liners into structured, queryable history. Changelogs write themselves. Version numbers bump automatically. Reviewers understand intent at a glance.

1. ANATOMY OF A COMMIT

Full format

type	(scope)	!	:	subject
required	optional	breaking		imperative mood, 72 chars max

Live examples

```
feat(auth)! add OAuth2 login with Google
fix(payments)! prevent double-charge on retry
feat(api)!: remove /v1/users endpoint ← breaking change
refactor(cache): extract TTL logic into helper
chore(deps): bump lodash from 4.17.20 to 4.17.21
docs(readme): add setup instructions for Windows
perf(db): batch inserts to reduce round-trips
test(auth): add edge cases for expired tokens
```

2. TYPE PREFIXES — COMPLETE REFERENCE

`feat`	**Feature**
	Introduces a new capability or behavior visible to users. This is the type that drives the product forward. Use feat whenever the end-user experience changes in an additive way — a new endpoint, a new UI component, a new configuration option.
	`feat(search): add fuzzy matching for product names feat(api): expose /v2/reports endpoint with pagination feat(ui): add dark mode toggle to settings panel`
`fix`	**Bug Fix**
	Corrects a defect in existing behavior. The key question is: was this working before and now it's broken? If yes, it's a fix. Even tiny fixes — off-by-one errors, wrong error messages, typos in validation — belong here. Don't use chore or refactor as a soft-pedal for fixes.
	`fix(auth): reject expired JWT tokens correctly fix(cart): prevent negative quantities on decrement fix(email): escape HTML entities in subject line`
`refactor`	**Refactor**
	Restructures existing code without changing observable behavior or adding features. The acid test: could you swap the old and new implementation in a production system with zero behavioral difference? Renaming variables, splitting large functions, moving modules, replacing loops with map/filter — all refactor. If behavior changes even slightly, use fix.
	`refactor(router): extract path-matching into RouteUtil refactor(db): replace raw SQL with ORM queries refactor(auth): simplify token validation chain`
`docs`	**Documentation**
	Changes that only touch documentation: READMEs, JSDoc/TSDoc comments, API docs, wiki pages, tutorials, changelogs written by hand, and inline code comments. If a single line of production logic also changed, reconsider whether it should be a different type with docs as the scope.
	`docs(api): document rate limiting headers docs(readme): add Docker setup instructions docs(contributing): clarify branching strategy`

`test`	**Tests**
	Adding missing tests or correcting broken ones. No production code changes allowed in a test commit — if you had to fix a bug to make a test pass, split into a fix commit and a test commit. Test infrastructure (fixtures, factories, test utilities) also goes here.
	`test(payments): add integration tests for Stripe webhooks test(auth): cover token refresh edge cases test(cache): add regression test for stale-read bug`

`chore`	**Chore**
	Maintenance work that doesn't fit build or ci — dependency bumps, generated file updates, license headers, git hooks, editor config. The defining characteristic: nothing a user of your software would notice or care about. When in doubt between chore and build/ci, prefer the more specific type.
	`chore(deps): bump axios from 1.3.4 to 1.4.0 chore: update .gitignore for JetBrains IDEs chore(licenses): add MIT header to new source files`

`perf`	**Performance**
	Improves speed, memory, or resource usage without changing observable behavior. Microbenchmarks, profiling-driven optimizations, caching layers, lazy loading, query index additions. Users may notice the app feels faster but their workflows are identical.
	`perf(images): lazy-load thumbnails below the fold perf(db): add index on orders.created_at perf(api): cache user permissions for 5 minutes`

`style`	**Style**
	Purely cosmetic changes that don't affect logic: whitespace, indentation, trailing commas, semicolons, import ordering. Usually generated by a formatter (Prettier, Black, gofmt). Run your formatter in CI and these commits become rare — but when they happen, label them honestly.
	`style: run Prettier on all TypeScript files style(components): normalize quote style to double style: remove trailing whitespace across the repo`

`revert`	**Revert**
	Undoes a previous commit. Always reference the reverted commit's subject in your message so the history is navigable. Git's revert command auto-generates this — just clean up the message to follow the format.
	`revert: feat(auth): add OAuth2 login with Google This reverts commit 3a2b1c9. Reason: caused session conflicts in multi-tenant setup`

`build`	**Build System**
	Changes that affect how the software is compiled, packaged, or published: webpack, Rollup, Vite, Makefile, CMake, Bazel, Dockerfile, package.json scripts. Distinct from ci — build is about producing the artifact, ci is about automating the pipeline that runs tests and ships it.
	`build: add tree-shaking to webpack config build(docker): switch base image to node:20-alpine build: enable source maps in production bundle`

`ci`	**Continuous Integration**
	Changes to CI/CD pipeline configuration: GitHub Actions, GitLab CI, CircleCI, Jenkins, Drone. Also covers deployment scripts, release automation, and environment config that lives in the repo but only runs in CI.
	`ci: add lint step to pull request workflow ci(deploy): switch to blue-green deployment ci: cache node_modules between workflow runs`

`wip`	**Work in Progress**
	Unfinished, checkpoint commits for saving progress. Never merge a wip commit to the main branch — always squash or amend before merging. Using wip consistently signals to teammates and tooling that the branch is not ready.
	`wip(payments): partial Stripe webhook handling wip: spike on WebSocket approach for live updates`

3. SCOPE — NAMING YOUR TARGET

The scope is an optional noun in parentheses that names the part of the codebase affected. Good scopes are short, stable, and match your project structure — they're the same words your team uses in conversation.

When to include scope

Include scope whenever the change is clearly localized to one subsystem and naming it saves reviewers time. Omit it for cross-cutting concerns (e.g. a repo-wide formatting pass) where listing every affected area would be noise.

Domain / feature areas

auth	payments	search	cart	checkout	notifications
reports	admin	billing			

Technical layers

api	db	cache	ui	cli	router
middleware	workers	queue			

Infrastructure

config	deps	ci	docker	k8s	logging
metrics	tracing				

Cross-functional

i18n	a11y	security	perf	types	core

Scope consistency tips

1. Define your team's allowed scope list in CONTRIBUTING.md and commit to it.
2. Match scopes to your directory structure — if the folder is 'services/payments', the scope is 'payments'.
3. Avoid generic scopes like 'misc', 'various', or 'stuff' — they defeat the purpose.
4. Monorepos often use the package name as scope: feat(web): ..., fix(api): ...
5. Tooling like commitlint can enforce an allowed-scopes allowlist automatically.

4. WRITING THE SUBJECT LINE

The subject is the single most-read part of your commit. It appears in git log, GitHub PR summaries, Slack notifications, and generated changelogs. Treat it like a headline — precise, scannable, and self-contained.

Imperative mood

Write the command the commit executes. Ask: 'If applied, this commit will...' and complete the sentence. 'add user avatar upload' not 'added user avatar upload' or 'adding user avatar upload'. This mirrors how git itself writes commits (Merge branch 'feature-x', Revert 'feat: ...').

```
add user avatar upload add dark mode to settings remove deprecated /v1 endpoints
```

72-character limit

Most terminal emulators, GitHub's commit list view, and email clients wrap or truncate beyond 72 characters. Count your characters — or configure your editor to show a ruler. If the subject is getting long, you're probably trying to describe two commits at once.

```
fix(auth): reject tokens with invalid signature <- 51 chars, good fix(auth): reject tokens with an invalid or malformed signature and refresh <- 78, too long
```

No period at the end

The subject is a title, not a sentence. Periods at the end look odd in git log output and don't add information. The same applies to exclamation marks used for emphasis (the ! breaking-change marker is different — it belongs before the colon, not at the end of the subject).

```
feat(cache): add Redis as session backend <- correct feat(cache): add Redis as session backend. <- incorrect
```

Lowercase after the colon

The subject begins in lowercase. This is a consistent visual signal that the subject has started. Your linter will catch violations. Exception: proper nouns and acronyms keep their casing: OAuth, GraphQL, AWS, iOS.

```
feat(auth): add OAuth2 support <- correct feat(auth): Add OAuth2 support <- incorrect (capital A)
```

What + why, not how

The subject tells readers what changed and why it matters — not how you implemented it. Save implementation details for the body. Compare: 'perf(db): reduce query time on large datasets' (good) vs 'perf(db): replace nested loop with hash join' (too implementation-focused).

```
fix(payments): prevent overcharge when coupon is applied perf(search): reduce p99 latency by 40% with result caching
```

5. BODY & FOOTER — THE FULL STORY

Most commits need only a subject line. But when the 'why' behind a change isn't obvious — a non-intuitive fix, a significant architectural decision, a workaround for a third-party bug — the body is where you explain it. Future you will be grateful.

Anatomy of a full commit message

```
fix(cache): prevent stale reads after TTL expiry

The cache was returning stale values for up to 30 seconds after
TTL because the expiry check ran BEFORE the system clock was read.
Moved the clock.Now() call to happen before the expiry comparison.

This was caught in production via elevated cache-miss alerts; the
stale window was causing users to see outdated pricing on the cart
page after a flash sale ended.

Closes: #1042
Tested-by: QA team (regression suite)
Co-authored-by: Ana Santos
```

Blank line separator

Separate the subject from the body with exactly one blank line. Git uses this blank line to distinguish the two sections. Many tools break if you omit it — git log --oneline will merge subject and body.

Explain the why, not the what

The diff shows what changed. The body should explain why: what problem existed, what alternatives were considered, what constraints forced this approach. Write for someone who has never seen this code.

72-character line wrapping in body

Wrap body lines at 72 characters too. This keeps commit messages readable in terminal views without horizontal scrolling.

Footer tokens

Footer tokens appear after a blank line following the body. Each token is on its own line: 'Closes: #N' (closes issue), 'Refs: #N' (references without closing), 'BREAKING CHANGE: description', 'Co-authored-by: Name '.

6. BREAKING CHANGES

A breaking change is any modification that requires consumers of your software to update their code, configuration, or workflows. This includes removing endpoints, renaming fields, changing default behavior, dropping platforms, and altering function signatures. Breaking changes trigger a major version bump in Semantic Versioning.

Two signals — use both

Signal	Where	Purpose
`!`	After the scope, before the colon	Human-readable flag in git log. Easy to spot when scanning history.
`BREAKING CHANGE:`	Footer of the commit body	Machine-readable token parsed by semantic-release and similar tools.

Full breaking change commit

```
feat(api)!: remove deprecated /v1/users endpoint

The /v1/users endpoint has been deprecated since v2.3.0 (6 months ago).
All clients should migrate to /v2/users, which returns the same data
with an added 'metadata' field and ISO 8601 timestamps.

BREAKING CHANGE: /v1/users is permanently removed. The endpoint
returned 410 Gone since v2.8.0 as a migration aid.

Migration guide: docs/migration/v1-to-v2.md
Closes: #892
```

What counts as a breaking change?

- ✗ Removing or renaming a public API endpoint, function, or class
- ✗ Changing a function's parameter order, names, or required/optional status
- ✗ Altering the format or schema of a response payload
- ✗ Dropping support for a language version, OS, or platform
- ✗ Changing the default value of a configuration option
- ✗ Requiring a new mandatory environment variable or config key
- ✗ Changing authentication or authorization behavior
- ✗ Removing a previously exported module, constant, or type

7. SEMANTIC VERSIONING IMPACT

Conventional Commits maps directly to Semantic Versioning (semver). Tools like semantic-release read your commit history and bump versions automatically — no manual version management needed.

Version bump	Triggered by	Example	Result
MAJOR	`!` marker or `BREAKING CHANGE:` footer	`feat(api)!: remove /v1`	`1.4.2 → 2.0.0`
MINOR	`feat` (non-breaking)	`feat(ui): add dark mode`	`1.4.2 → 1.5.0`
PATCH	`fix`, `perf`	`fix(cache): prevent stale reads`	`1.4.2 → 1.4.3`
NONE	`docs`, `chore`, `style`, `test`, `ci`, `refactor`	`chore(deps): bump lodash`	`1.4.2 → 1.4.2`

8. GOOD VS. BAD — 20 EXAMPLES

The fastest way to internalize the standard is to see the contrast. Each pair below shows the same intent expressed badly and well.

BAD	GOOD
`added google login stuff`	`feat(auth): add Google OAuth2 login`
`fix bug`	`fix(payments): prevent double-charge on retry`
`WIP`	`wip(search): spike on Elasticsearch integration`
`update packages`	`chore(deps): bump lodash from 4.17.20 to 4.17.21`
`made it faster`	`perf(db): batch inserts to reduce connection round-trips`
`formatting`	`style: run Prettier across all TypeScript files`
`fixed tests`	`test(auth): add coverage for expired-token edge cases`
`changes`	`refactor(router): extract path-matching into RouteUtil class`
`remove old stuff`	`chore: delete deprecated /v1 compatibility shims`
`docs`	`docs(api): document rate-limiting headers and retry behavior`
`deploy fix`	`ci: add health check before blue-green traffic switch`

updated docker	build(docker): switch base image to node:20-alpine
fixed the thing	fix(cart): correct quantity decrement on Safari iOS
new feature	feat(reports): add CSV export for invoice history
security	fix(auth): rotate JWT signing key on suspicious activity
refactored	refactor(db): replace raw SQL with Prisma ORM queries
added more tests	test(payments): add Stripe webhook integration tests
oops revert	revert: feat(cache): add Redis session backend
config update	chore(config): add ESLint rule for no-console in prod
small fix	fix(email): escape HTML entities in notification subject

9. TOOLING & AUTOMATION

The real payoff of semantic commits is automation. These tools read your commit history and do the work of versioning, changelog generation, and enforcement for you.

commitlint

Lints commit messages against the Conventional Commits spec. Runs as a git hook (via Husky) or in CI. Configure allowed types and scopes in commitlint.config.js.

```
npm install --save-dev @commitlint/{cli,config-conventional} husky
echo "module.exports = {extends: ['@commitlint/config-conventional']}" > commitlint.config.js
npx husky add .husky/commit-msg 'npx --no -- commitlint --edit "$1"'
```

semantic-release

Fully automated version management and package publishing. Reads your commits, determines the next semver, generates a changelog, creates a GitHub release, and publishes to npm — all triggered by a CI push.

```
npm install --save-dev semantic-release
# Add to .releaserc.json:
{ "branches": ["main"], "plugins": ["@semantic-release/commit-analyzer",
"@semantic-release/release-notes-generator", "@semantic-release/npm",
"@semantic-release/github"] }
```

standard-version

A simpler alternative to semantic-release. Bumps the version in package.json, generates a CHANGELOG.md, and creates a git tag. Run manually or in CI — doesn't auto-publish.

```
npm install --save-dev standard-version
# Add to package.json scripts:
"release": "standard-version"
npm run release # patch
npm run release -- --minor # force minor
```

Commitizen (cz-cli)

An interactive CLI prompt that guides contributors through writing a valid commit message — type, scope, subject, body, breaking changes. Great for teams adopting the standard.

```
npm install --save-dev commitizen cz-conventional-changelog
npx commitizen init cz-conventional-changelog --save-dev
git cz # instead of git commit
```

git-cliff

A fast changelog generator written in Rust. Highly configurable via cliff.toml. Can generate changelogs grouped by type, with custom templates and unreleased sections.

```
cargo install git-cliff
git cliff --init # generate cliff.toml
git cliff -o CHANGELOG.md # generate full changelog
git cliff --latest # since last tag only
```

10. ADOPTING IT AS A TEAM

The standard is only as valuable as its adoption. A single developer writing perfect commits in a repo full of 'fix stuff' messages won't unlock the tooling benefits.

1. Start with enforcement, not education

Install commitlint + Husky before announcing the change. The hook teaches by rejecting bad commits with a clear error message. Explaining the spec in a meeting is forgotten; a failed commit-msg hook is not.

2. Define your scope list

Add an allowed-scopes list to commitlint.config.js from day one. Without it, you'll end up with 'auth', 'Auth', 'authentication', and 'login' all meaning the same thing. Revisit and update the list as the project grows.

3. Squash merge feature branches

If developers commit freely during feature development, squash-merge into main so only one clean commit reaches the primary branch. This is the most pragmatic adoption path — developers don't need to think about message format until merge time.

4. Generate changelogs in CI

Add a step to your CI pipeline that generates the changelog and prints it in the PR description. Seeing 'Your commit will appear in the changelog as: feat(auth): add OAuth2 login' is a powerful feedback loop.

5. Don't audit old history

Draw a line and start from now. Retroactively reformatting thousands of commits is rarely worth the merge conflicts and history rewrite. Tag a version and start clean.

6. Add a CONTRIBUTING.md section

Document the commit format, your scope list, and the tooling setup. Link to this document in PR templates. New contributors should be able to get set up in under 5 minutes.

QUICK REFERENCE CARD

Print this page and keep it at your desk.

Type	When to use	Semver
`feat`	New user-visible feature	minor
`fix`	Bug correction	patch
`refactor`	Code restructure, no behavior change	none
`docs`	Documentation only	none
`test`	Add or fix tests	none
`chore`	Deps, tooling, config	none
`perf`	Speed / memory improvement	patch
`style`	Formatting, whitespace	none
`build`	Build system changes	none
`ci`	CI/CD pipeline changes	none
`revert`	Undo a previous commit	varies
`wip`	Work in progress (squash before merge)	none

Cheat sheet

```
Format: type(scope)!: subject

body (why, not what)

BREAKING CHANGE: description
Closes: #N

Subject: imperative mood · 72 chars · lowercase · no period
Scope: optional · noun · match your project structure
Breaking: ! before colon + BREAKING CHANGE: footer
Body: explain why · wrap at 72 chars · optional
```

Specification: conventionalcommits.org/en/v1.0.0 · Tooling: commitlint.js.org · semantic-release.gitbook.io